

Reporte

October 6, 2026

1 Laboratorio 8

1.1 Integrantes

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

1.2 Repositorio

Branch: lab8

<https://github.com/donmatthiuz/RL/tree/lab8>

1.3 Task 1

1.3.1 Task 1.1

Antes de implementar cualquier algoritmo, el equipo de ingeniería necesita una justificación técnica sólida de las decisiones de diseño.

Inciso a Pregunta

El espacio de acción del sistema de control de palas es continuo. Argumenten formalmente por qué DQN y sus variantes de la semana anterior son inapropiados para este dominio. Su argumento debe mencionar explícitamente el operador argmax, el espacio de acción continuo, y las consecuencias computacionales de intentar aplicar DQN directamente.

Respuesta

La acción del controlador es el ángulo de paso de cada pala. Si el ángulo de cada una de las tres palas se mueve en un rango $[\beta_{\min}, \beta_{\max}]$, el espacio de acción es

$$\mathcal{A} = [\beta_{\min}, \beta_{\max}]^3 \subset \mathbb{R}^3$$

Ese conjunto tiene infinitos elementos. DQN y sus variantes (Double DQN, Dueling DQN y PER) dependen del operador argmax en dos lugares. La política se obtiene como

$$\pi(s) = \arg \max_{a \in \mathcal{A}} Q(s, a; w)$$

y el target de cada actualización usa

$$y = r + \gamma \max_{a' \in \mathcal{A}} Q(s', a'; w^-)$$

Con cuatro acciones discretas el máximo sale de comparar cuatro salidas de la red. Con un espacio continuo la red ya no puede tener una neurona por acción y el máximo pasa a ser un problema de optimización sobre $\mathcal{A}$. Como $Q(s, \cdot; w)$ es una red neuronal, esa función tiene varios máximos locales y ningún método de búsqueda garantiza encontrar el global.

Hay dos formas de forzar DQN en este dominio y ambas fallan.

La primera es discretizar. Con k niveles por pala la capa de salida necesita k^3 neuronas. Una resolución de 0.1° en un rango de 0° a 90° da 901 niveles por pala y $901^3 \approx 7.3 \times 10^8$ salidas, una red imposible de entrenar con cualquier volumen de datos realista. Si se baja la resolución a unos pocos niveles el controlador solo puede dar saltos grandes de ángulo. Esos saltos cargan el actuador hidráulico y la estructura de la pala, y la potencia generada oscila. El error de discretización queda fijo por diseño y entrenar más no lo corrige.

La segunda es resolver el argmax de forma numérica en cada uso. Con ascenso por gradiente o con muestreo de N acciones candidatas cada decisión cuesta N evaluaciones de la red. Cada actualización cuesta $B \cdot N$ evaluaciones porque el máximo aparece en el target de cada una de las B transiciones del minibatch. Un controlador de paso decide decenas de veces por segundo, así que ese costo vuelve inviable el lazo de control. El máximo aproximado también contamina el aprendizaje. Si el optimizador se queda en un máximo local el target queda subestimado. Si se toman muchas muestras ruidosas el máximo selecciona errores positivos y la sobreestimación que estudiamos con Double DQN crece con N .

La salida es cambiar la representación de la política. En los métodos actor-crítico un actor $\mu_\theta(s)$ entrega la acción continua directamente en una sola evaluación, y el target se calcula con $Q(s', \mu_{\theta^-}(s'))$ sin ningún argmax. Esa es la estructura de DDPG, TD3 y SAC.

Inciso b Pregunta

Entre TD3 y SAC, ¿cuál recomendarían para el sistema de control de turbinas eólicas en producción? Justifiquen considerando: la naturaleza física del sistema (¿es deseable una política estocástica en un sistema de control industrial?), la eficiencia de datos durante el entrenamiento en simulación, y la robustez ante perturbaciones de viento no vistas durante el entrenamiento. Su respuesta debe argumentar ambos lados antes de tomar posición.

Respuesta

Los dos algoritmos comparten la base. Ambos son off-policy con replay buffer, usan dos críticos y toman el mínimo de ambos para frenar la sobreestimación. La diferencia está en la política. TD3 aprende una política determinista $\mu_\theta(s)$ y explora sumando ruido externo. SAC aprende una gaussiana $\pi_\theta(a | s)$ y maximiza la recompensa junto con la entropía:

$$J(\pi) = \sum_t \mathbb{E} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

A favor de TD3

Un sistema de control industrial debe responder igual ante el mismo estado. Una política estocástica en operación produce variaciones aleatorias del ángulo de paso. Cada variación mueve el actuador sin necesidad, acelera su desgaste y hace que la potencia entregada a la red oscile. TD3 produce ese comportamiento determinista de forma nativa. Un controlador determinista también es más fácil de verificar y certificar porque cada estado tiene una única acción que se puede auditar. TD3 tiene además menos piezas que ajustar. No hay temperatura α y su suavizado de la política objetivo ya da cierta tolerancia ante pequeños errores en la acción.

A favor de SAC

La estocasticidad de SAC sirve para entrenar y no tiene por qué llegar a producción. Al desplegar se usa la media de la gaussiana, $a = \tanh(\mu_\theta(s))$, y el controlador queda tan determinista como el de TD3. El argumento físico deja de ser una ventaja exclusiva de TD3.

En eficiencia de datos SAC suele aprender más rápido y con menos sensibilidad a hiperparámetros. La exploración se regula sola a través de la entropía con ajuste automático de α . En TD3 la escala del ruido de exploración se fija a mano y un mal valor deja al agente explorando de menos o golpeando los límites del actuador. Esto pesa en este dominio porque los simuladores aeroelásticos como OpenFAST son lentos y cada episodio simulado cuesta minutos.

En robustez SAC tiene el argumento más fuerte. Maximizar la entropía obliga al agente a mantener varias acciones buenas en cada estado en lugar de apostar todo a una. Eysenbach y Levine (ICLR 2022) demuestran que una política de máxima entropía es la solución de un problema robusto frente a ciertas perturbaciones de la dinámica y de la recompensa. Una ráfaga que el simulador nunca generó es justo una perturbación de la dinámica. TD3 optimiza para la distribución de viento del entrenamiento y puede quedar sobreajustado a ella. Su suavizado actúa sobre la acción y no da garantías sobre cambios en el entorno.

Posición

Recomendamos SAC para el entrenamiento en simulación y desplegar su acción media como controlador determinista. Así se obtiene la eficiencia y la robustez de SAC sin pagar el costo físico de la estocasticidad en operación. La suavidad no debe depender del algoritmo. En producción conviene limitar la tasa de cambio del ángulo de paso con un filtro o un límite de velocidad del actuador y penalizar $\|a_t - a_{t-1}\|^2$ en la recompensa. TD3 sería la opción si el regulador exigiera un controlador entrenado sin ningún componente aleatorio. La dejaríamos como alternativa de respaldo.

Inciso c Pregunta

La empresa tiene tres años de registros históricos generados por operadores humanos con distintos niveles de experiencia. Analicen si ese dataset es apropiado para aplicar CQL. Su análisis debe considerar: qué tipo de política de comportamiento β generó los datos (¿es cercana a la óptima?), qué cobertura del espacio estado-acción se puede esperar de operadores humanos, y cuál de las dos condiciones de Offline RL es más difícil de satisfacer en este dominio específico.

Respuesta

La política de comportamiento es una mezcla de las políticas de cada operador:

$$\beta(a | s) = \sum_k w_k \pi_k(a | s)$$

donde w_k es la fracción de datos que aportó el operador k . Esa mezcla está lejos de la óptima. Los operadores novatos cometen errores y tardan en reaccionar. Los expertos operan bien pero con márgenes amplios de seguridad que sacrifican potencia. Además operadores distintos responden distinto ante el mismo estado, así que β es multimodal. CQL tolera bien un β subóptimo porque está diseñado para mejorar sobre los datos sin salir de su soporte. Lo que no puede hacer es inventar buen comportamiento donde no hay registros, así que su techo queda cerca de lo que hicieron los mejores operadores.

La cobertura esperada es estrecha. Un operador humano repite los mismos procedimientos y casi nunca prueba acciones fuera de lo habitual. En un mismo estado las acciones registradas se concentran en una banda angosta y $\beta(a | s)$ vale casi cero en el resto de $\mathcal{A}$. En el espacio de estados la cobertura es buena para el viento normal porque tres años incluyen todas las estaciones. Los estados raros están poco representados. Ráfagas extremas, vientos sobre la velocidad de corte y fallas parciales de sensores aparecen pocas veces, y en esos casos el operador suele detener la turbina. Esa respuesta deja todavía menos variedad de acciones registradas.

Offline RL necesita dos condiciones. La primera es que los datos tengan suficiente calidad, es decir que β contenga comportamiento razonablemente bueno. La segunda es que cubran los pares estado-acción que visitaría la política que se quiere aprender. En este dominio la condición difícil es la cobertura. La calidad se puede manejar porque hay operadores expertos y se pueden filtrar o ponderar sus trayectorias. La cobertura no mejora con más años de registros, ya que los mismos operadores seguirán tomando las mismas decisiones. Cualquier acción mejor que la humana que nadie probó queda fuera del alcance de CQL por construcción.

El dataset sí es apropiado para CQL como punto de partida. La política resultante será parecida a la de los operadores más experimentados y algo más consistente que ellos. Para pasar de ese nivel hace falta complementar los datos con simulación o con exploración controlada en línea.

Inciso d Pregunta

Argumenten cuál es el riesgo principal de aplicar Q-Learning estándar directamente sobre el dataset histórico sin ninguna restricción de conservatividad. Usen el concepto de error de extrapolación y conéctenlo con una consecuencia operacional concreta para el sistema de turbinas.

Respuesta

Q-Learning estándar actualiza con el target

$$y = r + \gamma \max_{a'} Q(s', a'; w)$$

El máximo recorre todas las acciones posibles, incluidas las que nunca aparecen en el dataset. Para esas acciones la red no recibió ninguna señal de entrenamiento y su valor es pura extrapolación de la red. Ese es el error de extrapolación. En RL en línea el error se corrige solo, porque el agente prueba la acción sobreestimada, observa una recompensa mala y ajusta el valor. Offline esa corrección no existe porque el agente nunca interactúa con la turbina.

El operador max empeora el problema. Entre muchas acciones con errores aleatorios elige la que tiene el error positivo más grande, así que el target se infla justo con valores que no tienen respaldo en los datos. El bootstrapping propaga ese valor inflado hacia los estados anteriores y los valores Q pueden crecer sin control. La política aprendida termina eligiendo las acciones fuera de distribución porque son las que la red sobrevalora.

Un caso concreto en la turbina es la operación sobre la velocidad nominal del viento. En esa región el controlador debe aumentar el ángulo de paso para limitar la potencia y la velocidad del rotor. Los operadores siempre lo hicieron, así que el dataset casi no contiene registros de palas en ángulo bajo con viento fuerte. Q-Learning no tiene datos que digan que esa acción es peligrosa y puede extrapolar que mantener el ángulo bajo captura más energía y vale más. Al desplegar esa política el rotor se acelera por encima de su límite, las cargas sobre las palas y la torre crecen y se dispara un paro de emergencia. En el peor caso hay daño estructural en la pala o en la caja de engranajes. El riesgo principal es entonces una política que confía en acciones que nadie probó. La restricción de conservatividad de CQL ataca justo eso porque baja el valor Q de las acciones fuera del dataset y lo sube en las acciones registradas.

1.3.2 Task 1.2

El equipo quiere entender las diferencias matemáticas entre los algoritmos antes de implementarlos.

Inciso a Pregunta

Escriban el objetivo de optimización de SAC incluyendo el término de entropía. Identifiquen cada variable con su significado en el contexto del sistema de turbinas: ¿qué representa α , qué representa $\mathcal{H}(\pi(\cdot \mid S_t))$, y cómo se calibraría α automáticamente durante el entrenamiento?

Respuesta

SAC maximiza la recompensa esperada junto con la entropía de la política en cada estado visitado:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(S_t, A_t) \sim \rho_\pi} \left[r(S_t, A_t) + \alpha \mathcal{H}(\pi(\cdot \mid S_t)) \right]$$

con la entropía definida como

$$\mathcal{H}(\pi(\cdot \mid S_t)) = -\mathbb{E}_{a \sim \pi(\cdot \mid S_t)} [\log \pi(a \mid S_t)]$$

Variable	Significado en la turbina
S_t	Estado medido: velocidad del viento, velocidad del rotor, potencia generada, ángulo de paso actual y cargas en palas y torre
A_t	Comando de ángulo de paso para las palas
$r(S_t, A_t)$	Recompensa: seguimiento de la potencia nominal menos penalizaciones por cargas estructurales y por movimiento del actuador
ρ_π	Distribución de estados y acciones que visita la turbina bajo la política π

Variable	Significado en la turbina
$\pi(\cdot \mid S_t)$	Distribución gaussiana sobre los ángulos de paso dado el estado
$\mathcal{H}(\pi(\cdot \mid S_t))$	Qué tan dispersos son los ángulos que la política considera buenos en ese estado
α	Temperatura: cuánta recompensa está dispuesto a ceder el agente a cambio de seguir explorando

$\mathcal{H}(\pi(\cdot \mid S_t))$ mide la incertidumbre del controlador sobre qué ángulo usar. Una entropía alta significa que el agente todavía prueba varios ángulos en esa condición de viento. Una entropía baja significa que ya se decidió por un ángulo concreto. α fija el precio de esa exploración. Con α grande el agente sacrifica potencia para seguir probando ángulos distintos. Con α cercano a cero el objetivo se reduce al de RL estándar y la política se vuelve casi determinista.

Fijar α a mano es difícil porque la escala de la recompensa cambia entre regiones de operación. Por eso SAC ajusta α como un multiplicador de Lagrange que obliga a la política a mantener una entropía mínima $\bar{\mathcal{H}}$. Se minimiza

$$J(\alpha) = \mathbb{E}_{S_t \sim \mathcal{D}, a \sim \pi(\cdot \mid S_t)} \left[-\alpha (\log \pi(a \mid S_t) + \bar{\mathcal{H}}) \right]$$

La entropía objetivo suele fijarse en $\bar{\mathcal{H}} = -\dim(\mathcal{A})$, que con tres palas da -3 . El gradiente tiene una lectura directa. Si la entropía actual cae por debajo de $\bar{\mathcal{H}}$, α sube y empuja al agente a explorar más. Si está por encima, α baja y deja que la política se concentre. En la práctica se optimiza $\log \alpha$ para que α siempre sea positivo. Al inicio del entrenamiento α es alto y la turbina simulada prueba muchos ángulos. Conforme el crítico mejora α decrece y la política converge a un controlador preciso.

Inciso b Pregunta

Comparen el target de actualización del Critic en TD3 versus SAC. ¿Cuál de los dos produce estimaciones más conservadoras del valor y por qué? ¿Esa conservatividad es una ventaja o una desventaja para el dominio de turbinas?

Respuesta

El target de TD3 es

$$y_{\text{TD3}} = r + \gamma \min_{i=1,2} Q_{\phi'_i}(s', \tilde{a}'), \quad \tilde{a}' = \text{clip}(\mu_{\theta'}(s') + \text{clip}(\epsilon, -c, c), a_{\text{low}}, a_{\text{high}}), \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

El target de SAC es

$$y_{\text{SAC}} = r + \gamma \left(\min_{i=1,2} Q_{\phi'_i}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}' \mid s') \right), \quad \tilde{a}' \sim \pi_{\theta}(\cdot \mid s')$$

Los dos toman el mínimo de dos críticos. Ese mínimo ya introduce un sesgo hacia abajo porque entre dos estimaciones ruidosas se queda con la menor. Las diferencias están en la acción siguiente y en el término de entropía.

TD3 evalúa la acción de una política determinista con un ruido pequeño y recortado. SAC evalúa una acción muestreada de la política actual y suma el término $-\alpha \log \pi_\theta(\tilde{a}' | s')$. En valor esperado ese término es $\alpha \mathcal{H}(\pi(\cdot | s'))$, un bono que premia llegar a estados donde la política todavía tiene incertidumbre. El crítico de SAC estima el valor suave, que es la recompensa futura más la entropía futura. Mientras α sea grande ese bono infla el valor por encima de la recompensa real que obtendría la turbina.

TD3 produce las estimaciones más conservadoras. Combina el mínimo de dos críticos con una política determinista sin ningún bono de exploración. Fujimoto y coautores reportan que ese diseño tiende incluso a subestimar el valor verdadero. En SAC la conservatividad del mínimo queda compensada en parte por el bono de entropía, que se desvanece conforme el ajuste automático baja α .

Para la turbina la conservatividad tiene dos caras. Del lado positivo un crítico que subestima no confía en acciones agresivas cerca de los límites del rotor. En un sistema donde un error cuesta una pala dañada es preferible equivocarse hacia la prudencia. Del lado negativo un crítico demasiado pesimista frena el aprendizaje y deja una política tímida que lleva el ángulo de paso hacia afuera antes de lo necesario. Esa política pierde energía en la región de operación nominal donde está casi toda la producción anual. La conservatividad moderada es una ventaja en este dominio y la excesiva se vuelve un costo económico directo. SAC ofrece el punto intermedio porque su bono de entropía compensa el pesimismo durante la exploración y desaparece al final, cuando la política ya es precisa.

Inciso c Pregunta

Escriban el objetivo de CQL identificando los dos términos que se añaden a la pérdida TD estándar. Expliquen en palabras qué hace cada término y por qué su combinación produce un Critic conservador. Luego argumenten qué valor de α_{CQL} recomendarían para el dataset de turbinas considerando la heterogeneidad de los operadores que lo generaron.

Respuesta

El objetivo de CQL para el crítico es

$$\min_Q \alpha_{CQL} \left(\underbrace{\mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_a \exp Q(s, a) \right]}_{\text{término 1}} - \underbrace{\mathbb{E}_{(s,a) \sim \mathcal{D}} [Q(s, a)]}_{\text{término 2}} \right) + \frac{1}{2} \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[(Q(s, a) - \hat{\mathcal{B}}^\pi \hat{Q}(s, a))^2 \right]$$

El último sumando es la pérdida TD estándar. Los dos términos nuevos van multiplicados por α_{CQL} . Con acciones continuas la suma sobre a se reemplaza por una integral que se estima con muestras de una distribución uniforme y de la política actual.

El término 1 es un máximo suave sobre todas las acciones. Minimizarlo baja el valor Q de las acciones a las que el crítico les asigna valores altos. Afecta sobre todo a las acciones que no están en el dataset, porque allí la red puede inventar valores grandes sin que ningún dato lo impida.

El término 2 entra con signo negativo. Minimizar $-Q(s, a)$ sube el valor de las acciones que sí aparecen en el dataset y compensa el empuje hacia abajo del término 1 en el soporte de los datos.

Juntos crean una brecha. Las acciones registradas por los operadores conservan su valor y las acciones que nadie probó quedan penalizadas. Kumar y coautores demuestran que con α_{CQL} suficientemente grande el valor aprendido de la política es una cota inferior del valor verdadero. La política que maximiza ese crítico prefiere quedarse dentro del soporte del dataset, que es justo la protección contra el error de extrapolación descrito en la Task 1.1d.

Para elegir α_{CQL} en las turbinas pesan dos efectos de la heterogeneidad de los operadores. El primero es que el dataset contiene acciones malas de operadores novatos. El término 2 sube el valor de todas las acciones registradas sin distinguir su calidad, así que un α_{CQL} muy alto ata la política al comportamiento promedio, errores incluidos. El segundo es que la cobertura de cada estado es angosta, como se argumentó en la Task 1.1c. Un α_{CQL} bajo deja espacio para que la política elija ángulos que ningún operador probó, que es el riesgo operacional más grave.

Como el error de extrapolación puede dañar la turbina y el apego a los datos solo cuesta eficiencia, recomendamos empezar con un valor alto. Proponemos $\alpha_{CQL} = 5$, el valor que Kumar y coautores usan en tareas de control continuo. Lo combinaríamos con la variante de Lagrange de CQL, que ajusta α_{CQL} de forma automática para mantener la brecha entre ambos términos cerca de un umbral τ . Ese ajuste evita fijar el valor a ciegas. Antes de entrenar filtraríamos o ponderaríamos las trayectorias según la experiencia del operador, así el término 2 favorece a los expertos. El valor final se validaría con evaluación off-policy sobre un año de registros reservado y luego en simulación antes de tocar una turbina real.

2 Task 2

2.1 Task 2.1

Implementen TD3 y SAC sobre el entorno Pendulum-v1 de Gymnasium como proxy del sistema de control de turbinas. Este entorno tiene espacio de acción continuo unidimensional en $[-2, 2]$ y estado tridimensional, lo que lo hace apropiado para validar las propiedades algorítmicas antes de escalar al dominio real.

Ambos algoritmos deben implementarse desde cero usando PyTorch. No se permite usar implementaciones preconstruidas de TD3 o SAC de ninguna librería de RL. Usen las siguientes especificaciones compartidas para ambos: buffer de Experience Replay con capacidad de 100,000 transiciones, minibatch de 256 transiciones, Target Networks con actualización suave $\tau = 0.005$, dos capas ocultas de 256 neuronas con activación ReLU, y entrenamiento durante 100,000 pasos de ambiente.

Para TD3 específicamente: Twin Critics, Policy Delay de 2 pasos, y Target Policy Smoothing con $\sigma = 0.2$ y clip $c = 0.5$.

Para SAC específicamente: política Gaussiana reparametrizada con tanh aplicado a la salida para restringir acciones al rango válido, Twin Critics, y α ajustado automáticamente con entropía objetivo $-\dim(A)$.

La implementación debe registrar por cada 1,000 pasos: recompensa promedio de evaluación greedy sobre 10 episodios, valor Q promedio estimado sobre un conjunto fijo de 1,000 transiciones, y entropía de la política (solo para SAC).

```
[23]: import random
      from collections import deque

      import gymnasium as gym
      import numpy as np

      import torch
      import torch.nn as nn
      import torch.nn.functional as F
      import torch.optim as optim
```

```
[24]: SEED = 42

      # Compartidos
      BUFFER_SIZE = 100_000
      BATCH_SIZE = 256
      TAU = 0.005
      TOTAL_STEPS = 100_000
      EVAL_INTERVAL = 1_000
      EVAL_EPISODES = 10
      FIXED_Q_SAMPLES = 1_000

      # TD3
      POLICY_DELAY = 2
      TARGET_NOISE = 0.2
      NOISE_CLIP = 0.5

      # Reproducibilidad
      random.seed(SEED)
      np.random.seed(SEED)
      torch.manual_seed(SEED)

      device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

      print("Device:", device)
```

Device: cpu

```
[25]: env = gym.make("Pendulum-v1")

      obs, _ = env.reset(seed=SEED)
      env.action_space.seed(SEED)

      state_dim = env.observation_space.shape[0]
      action_dim = env.action_space.shape[0]

      print("Estado inicial:", obs)
      print("Dimensión del estado:", state_dim)
```

```
print("Dimensión de la acción:", action_dim)
print("Límite inferior:", env.action_space.low)
print("Límite superior:", env.action_space.high)
```

Estado inicial: [-0.14995256 0.9886932 -0.12224312]

Dimensión del estado: 3

Dimensión de la acción: 1

Límite inferior: [-2.]

Límite superior: [2.]

```
[26]: class ReplayBuffer:
    def __init__(self, capacity=BUFFER_SIZE):
        self.buffer = deque(maxlen=capacity)

    def add(self, state, action, reward, next_state, terminated):
        self.buffer.append((
            np.asarray(state, dtype=np.float32),
            np.asarray(action, dtype=np.float32),
            np.float32(reward),
            np.asarray(next_state, dtype=np.float32),
            np.float32(terminated),
        ))

    def sample(self, batch_size=BATCH_SIZE):
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, terminateds = zip(*batch)

        states = torch.as_tensor(
            np.asarray(states), dtype=torch.float32, device=device
        )
        actions = torch.as_tensor(
            np.asarray(actions), dtype=torch.float32, device=device
        )
        rewards = torch.as_tensor(
            np.asarray(rewards), dtype=torch.float32, device=device
        ).unsqueeze(1)
        next_states = torch.as_tensor(
            np.asarray(next_states), dtype=torch.float32, device=device
        )
        terminateds = torch.as_tensor(
            np.asarray(terminateds), dtype=torch.float32, device=device
        ).unsqueeze(1)

        return states, actions, rewards, next_states, terminateds

    def __len__(self):
        return len(self.buffer)
```

```
[27]: replay_buffer = ReplayBuffer()

state, _ = env.reset(seed=SEED)

action = env.action_space.sample()
next_state, reward, terminated, truncated, _ = env.step(action)

done = terminated or truncated

replay_buffer.add(
    state,
    action,
    reward,
    next_state,
    done
)

print("Transiciones guardadas:", len(replay_buffer))
print("Acción:", action)
print("Reward:", reward)
```

```
Transiciones guardadas: 1
Acción: [1.0958242]
Reward: -2.965626072153192
```

Se obtuvo un estado de dimensión 3 y una acción continua en $[-2, 2]$. Además, el Experience Replay almacenó correctamente una transición, estableciendo la infraestructura base para el entrenamiento posterior de **TD3** y **SAC**.

2.2 Task 2.2

Generen el dataset para el experimento de Offline RL entrenando primero una política subóptima sobre Pendulum-v1: entrenen TD3 durante solo 20,000 pasos (sin convergencia completa) y recolecten 50,000 transiciones con esa política más exploración adicional con ruido gaussiano $\mathcal{N}(0, 0.3)$. Este dataset simula los registros históricos de operadores humanos con desempeño heterogéneo.

Implementen CQL sobre ese dataset. Usen la misma arquitectura de red que en Task 2.1. La pérdida de CQL debe incluir el término TD estándar más el término de regularización con $\alpha_{CQL} \in \{0.5, 1.0, 5.0\}$. Entrenen durante 100,000 actualizaciones sobre el dataset sin ninguna interacción adicional con el entorno.

Evalúen la política aprendida por CQL ejecutándola sobre el entorno real durante 10 episodios por cada valor de α_{CQL} . Comparen esa evaluación con la recompensa promedio de la política subóptima que generó el dataset.

```
[28]: GAMMA = 0.99
      ACTOR_LR = 3e-4
      CRITIC_LR = 3e-4
```

```

SUBOPTIMAL_STEPS = 20_000
LEARNING_STARTS = 1_000
EXPLORATION_NOISE = 0.1

action_low = float(env.action_space.low[0])
action_high = float(env.action_space.high[0])
max_action = action_high

```

```

[29]: class Actor(nn.Module):
    def __init__(self, state_dim, action_dim, max_action):
        super().__init__()

        self.fc1 = nn.Linear(state_dim, 256)
        self.fc2 = nn.Linear(256, 256)
        self.fc3 = nn.Linear(256, action_dim)

        self.max_action = max_action

    def forward(self, state):
        x = F.relu(self.fc1(state))
        x = F.relu(self.fc2(x))
        return self.max_action * torch.tanh(self.fc3(x))

```

```

[30]: class Critic(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()

        # Q1
        self.q1_fc1 = nn.Linear(state_dim + action_dim, 256)
        self.q1_fc2 = nn.Linear(256, 256)
        self.q1_fc3 = nn.Linear(256, 1)

        # Q2
        self.q2_fc1 = nn.Linear(state_dim + action_dim, 256)
        self.q2_fc2 = nn.Linear(256, 256)
        self.q2_fc3 = nn.Linear(256, 1)

    def forward(self, state, action):
        sa = torch.cat([state, action], dim=1)

        q1 = F.relu(self.q1_fc1(sa))
        q1 = F.relu(self.q1_fc2(q1))
        q1 = self.q1_fc3(q1)

        q2 = F.relu(self.q2_fc1(sa))
        q2 = F.relu(self.q2_fc2(q2))
        q2 = self.q2_fc3(q2)

```

```

        return q1, q2

    def q1(self, state, action):
        sa = torch.cat([state, action], dim=1)

        q1 = F.relu(self.q1_fc1(sa))
        q1 = F.relu(self.q1_fc2(q1))
        q1 = self.q1_fc3(q1)

        return q1

```

```

[31]: def soft_update(source, target, tau=TAU):
    for source_param, target_param in zip(
        source.parameters(), target.parameters()
    ):
        target_param.data.copy_(
            tau * source_param.data
            + (1.0 - tau) * target_param.data
        )

```

```

[ ]: class TD3:
    def __init__(self, state_dim, action_dim, max_action):
        self.max_action = max_action

        # Actor y Target Actor
        self.actor = Actor(
            state_dim, action_dim, max_action
        ).to(device)

        self.actor_target = Actor(
            state_dim, action_dim, max_action
        ).to(device)

        self.actor_target.load_state_dict(
            self.actor.state_dict()
        )

        self.actor_optimizer = optim.Adam(
            self.actor.parameters(),
            lr=ACTOR_LR
        )

        # Twin Critics y Target Critics
        self.critic = Critic(
            state_dim, action_dim
        ).to(device)

```

```

self.critic_target = Critic(
    state_dim, action_dim
).to(device)

self.critic_target.load_state_dict(
    self.critic.state_dict()
)

self.critic_optimizer = optim.Adam(
    self.critic.parameters(),
    lr=CRITIC_LR
)

self.total_updates = 0
def select_action(self, state):
    state = torch.as_tensor(
        state,
        dtype=torch.float32,
        device=device
    ).unsqueeze(0)

    with torch.no_grad():
        action = self.actor(state)

    return action.cpu().numpy()[0]

def train(self, replay_buffer):
    self.total_updates += 1

    states, actions, rewards, next_states, terminateds = (
        replay_buffer.sample(BATCH_SIZE)
    )

    # -----
    # Target del Critic
    # -----
    with torch.no_grad():

        # Target Policy Smoothing
        noise = torch.randn_like(actions) * TARGET_NOISE
        noise = noise.clamp(
            -NOISE_CLIP,
            NOISE_CLIP
        )

        next_actions = (

```

```

        self.actor_target(next_states) + noise
    ).clamp(
        -self.max_action,
        self.max_action
    )

    target_q1, target_q2 = self.critic_target(
        next_states,
        next_actions
    )

    target_q = torch.min(
        target_q1,
        target_q2
    )

    target = rewards + (
        GAMMA
        * (1.0 - terminateds)
        * target_q
    )

# -----
# Actualización Twin Critics
# -----
    current_q1, current_q2 = self.critic(
        states,
        actions
    )

    critic_loss = (
        F.mse_loss(current_q1, target)
        + F.mse_loss(current_q2, target)
    )

    self.critic_optimizer.zero_grad()
    critic_loss.backward()
    self.critic_optimizer.step()

# -----
# Delayed Policy Update
# -----
    if self.total_updates % POLICY_DELAY == 0:

        actor_actions = self.actor(states)

        actor_loss = -self.critic.q1(

```

```

        states,
        actor_actions
    ).mean()

    self.actor_optimizer.zero_grad()
    actor_loss.backward()
    self.actor_optimizer.step()

    # Target Networks
    soft_update(
        self.actor,
        self.actor_target
    )

    soft_update(
        self.critic,
        self.critic_target
    )

```

```

[33]: td3_suboptimal = TD3(
        state_dim=state_dim,
        action_dim=action_dim,
        max_action=max_action
    )

    td3_buffer = ReplayBuffer()

    print(td3_suboptimal.actor)

```

```

Actor(
  (fc1): Linear(in_features=3, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=256, bias=True)
  (fc3): Linear(in_features=256, out_features=1, bias=True)
)

```

```

[34]: state, _ = env.reset(seed=SEED)

    episode_reward = 0.0
    episode_rewards = []

    for step in range(1, SUBOPTIMAL_STEPS + 1):

        if step <= LEARNING_STARTS:
            action = env.action_space.sample()

        else:
            action = td3_suboptimal.select_action(state)

```

```

    noise = np.random.normal(
        0,
        EXPLORATION_NOISE,
        size=action_dim
    )

    action = np.clip(
        action + noise,
        action_low,
        action_high
    )

    next_state, reward, terminated, truncated, _ = (
        env.step(action)
    )

    # Guardamos únicamente terminated para el bootstrap.
    td3_buffer.add(
        state,
        action,
        reward,
        next_state,
        terminated
    )

    state = next_state
    episode_reward += reward

    if (
        step > LEARNING_STARTS
        and len(td3_buffer) >= BATCH_SIZE
    ):
        td3_suboptimal.train(td3_buffer)

    # terminated o truncated si requieren reset del entorno.
    if terminated or truncated:
        episode_rewards.append(episode_reward)
        state, _ = env.reset()
        episode_reward = 0.0

    if step % 1_000 == 0:
        recent_reward = (
            np.mean(episode_rewards[-10:])
            if episode_rewards
            else np.nan
        )

```

```

    print(
        f"Step {step:>5} | "
        f"Buffer {len(td3_buffer):>5} | "
        f"Reward reciente {recent_reward:.2f}"
    )

print("Pasos de entrenamiento:", SUBOPTIMAL_STEPS)
print("Transiciones almacenadas:", len(td3_buffer))
print("Episodios completados:", len(episode_rewards))

```

```

Step 1000 | Buffer 1000 | Reward reciente -1378.91
Step 2000 | Buffer 2000 | Reward reciente -1369.28
Step 3000 | Buffer 3000 | Reward reciente -1419.59
Step 4000 | Buffer 4000 | Reward reciente -1401.39
Step 5000 | Buffer 5000 | Reward reciente -1162.84
Step 6000 | Buffer 6000 | Reward reciente -900.43
Step 7000 | Buffer 7000 | Reward reciente -717.94
Step 8000 | Buffer 8000 | Reward reciente -499.92
Step 9000 | Buffer 9000 | Reward reciente -281.58
Step 10000 | Buffer 10000 | Reward reciente -190.03
Step 11000 | Buffer 11000 | Reward reciente -176.15
Step 12000 | Buffer 12000 | Reward reciente -137.11
Step 13000 | Buffer 13000 | Reward reciente -146.92
Step 14000 | Buffer 14000 | Reward reciente -156.81
Step 15000 | Buffer 15000 | Reward reciente -244.17
Step 16000 | Buffer 16000 | Reward reciente -398.44
Step 17000 | Buffer 17000 | Reward reciente -276.81
Step 18000 | Buffer 18000 | Reward reciente -122.56
Step 19000 | Buffer 19000 | Reward reciente -120.80
Step 20000 | Buffer 20000 | Reward reciente -163.66
Pasos de entrenamiento: 20000
Transiciones almacenadas: 20000
Episodios completados: 100

```

```

[35]: def evaluate_policy(policy, episodes=EVAL_EPISODES, seed=SEED + 100):
    eval_env = gym.make("Pendulum-v1")
    rewards = []

    for episode in range(episodes):
        state, _ = eval_env.reset(seed=seed + episode)
        episode_reward = 0.0

        while True:
            action = policy.select_action(state)

            state, reward, terminated, truncated, _ = (
                eval_env.step(action)

```

```

    )

    episode_reward += reward

    if terminated or truncated:
        break

    rewards.append(episode_reward)

eval_env.close()

return np.mean(rewards), np.std(rewards)

suboptimal_mean, suboptimal_std = evaluate_policy(
    td3_suboptimal
)

print(
    f"TD3 subóptimo: "
    f"{suboptimal_mean:.2f} ± {suboptimal_std:.2f}"
)

```

TD3 subóptimo: -163.31 ± 115.00

```

[36]: OFFLINE_DATASET_SIZE = 50_000
      DATASET_NOISE = 0.3

      offline_buffer = ReplayBuffer(
          capacity=OFFLINE_DATASET_SIZE
      )

      dataset_env = gym.make("Pendulum-v1")
      dataset_env.action_space.seed(SEED + 200)

      state, _ = dataset_env.reset(seed=SEED + 200)

      dataset_episode_rewards = []
      episode_reward = 0.0

      for _ in range(OFFLINE_DATASET_SIZE):
          action = td3_suboptimal.select_action(state)

          noise = np.random.normal(
              0.0,
              DATASET_NOISE,
              size=action_dim

```

```

    )

    noisy_action = np.clip(
        action + noise,
        action_low,
        action_high
    ).astype(np.float32)

    next_state, reward, terminated, truncated, _ = (
        dataset_env.step(noisy_action)
    )

    offline_buffer.add(
        state,
        noisy_action,
        reward,
        next_state,
        terminated
    )

    episode_reward += reward
    state = next_state

    if terminated or truncated:
        dataset_episode_rewards.append(episode_reward)
        state, _ = dataset_env.reset()
        episode_reward = 0.0

dataset_env.close()

print("Tamaño dataset:", len(offline_buffer))
print("Episodios:", len(dataset_episode_rewards))
print(
    "Reward medio durante recolección:",
    np.mean(dataset_episode_rewards)
)

```

Tamaño dataset: 50000

Episodios: 250

Reward medio durante recolección: -159.80633766616717

```

[37]: CQL_LR = 3e-4
      CQL_UPDATES = 100_000
      CQL_NUM_RANDOM = 10
      CQL_ALPHAS = [0.5, 1.0, 5.0]

```

```

[38]: class CQL:
    def __init__(
        self,
        state_dim,
        action_dim,
        max_action,
        alpha_cql
    ):
        self.state_dim = state_dim
        self.action_dim = action_dim
        self.max_action = max_action
        self.alpha_cql = alpha_cql

        self.actor = Actor(
            state_dim,
            action_dim,
            max_action
        ).to(device)

        self.actor_target = Actor(
            state_dim,
            action_dim,
            max_action
        ).to(device)

        self.actor_target.load_state_dict(
            self.actor.state_dict()
        )

        self.critic = Critic(
            state_dim,
            action_dim
        ).to(device)

        self.critic_target = Critic(
            state_dim,
            action_dim
        ).to(device)

        self.critic_target.load_state_dict(
            self.critic.state_dict()
        )

        self.actor_optimizer = optim.Adam(
            self.actor.parameters(),
            lr=CQL_LR
        )

```

```

        self.critic_optimizer = optim.Adam(
            self.critic.parameters(),
            lr=CQL_LR
        )

    def select_action(self, state):
        state = torch.as_tensor(
            state,
            dtype=torch.float32,
            device=device
        ).unsqueeze(0)

        with torch.no_grad():
            action = self.actor(state)

        return action.cpu().numpy()[0]

    def train(self, replay_buffer):
        states, actions, rewards, next_states, terminateds = (
            replay_buffer.sample(BATCH_SIZE)
        )

        # TD target
        with torch.no_grad():
            next_actions = self.actor_target(next_states)

            target_q1, target_q2 = self.critic_target(
                next_states,
                next_actions
            )

            target_q = torch.min(
                target_q1,
                target_q2
            )

            target = (
                rewards
                + GAMMA
                * (1.0 - terminateds)
                * target_q
            )

        current_q1, current_q2 = self.critic(
            states,
            actions

```

```

)

td_loss = (
    F.mse_loss(current_q1, target)
    + F.mse_loss(current_q2, target)
)

# CQL conservative penalty
batch_size = states.shape[0]

repeated_states = (
    states.unsqueeze(1)
    .expand(-1, CQL_NUM_RANDOM, -1)
    .reshape(
        batch_size * CQL_NUM_RANDOM,
        self.state_dim
    )
)

random_actions = torch.empty(
    batch_size * CQL_NUM_RANDOM,
    self.action_dim,
    device=device
).uniform_(
    -self.max_action,
    self.max_action
)

random_q1, random_q2 = self.critic(
    repeated_states,
    random_actions
)

random_q1 = random_q1.reshape(
    batch_size,
    CQL_NUM_RANDOM
)

random_q2 = random_q2.reshape(
    batch_size,
    CQL_NUM_RANDOM
)

cql1 = (
    torch.logsumexp(random_q1, dim=1).mean()
    - current_q1.mean()
)

```

```

cql2 = (
    torch.logsumexp(random_q2, dim=1).mean()
    - current_q2.mean()
)

cql_penalty = cql1 + cql2

critic_loss = (
    td_loss
    + self.alpha_cql * cql_penalty
)

self.critic_optimizer.zero_grad()
critic_loss.backward()
self.critic_optimizer.step()

# Actor
actor_actions = self.actor(states)

actor_loss = -self.critic.q1(
    states,
    actor_actions
).mean()

self.actor_optimizer.zero_grad()
actor_loss.backward()
self.actor_optimizer.step()

# Target networks
soft_update(
    self.actor,
    self.actor_target
)

soft_update(
    self.critic,
    self.critic_target
)

return (
    critic_loss.item(),
    td_loss.item(),
    cql_penalty.item()
)

```

```

[39]: cql_agents = {}
      cql_training_logs = {}

      for alpha in CQL_ALPHAS:
          print(f"\n===== CQL alpha={alpha} =====")

          # Misma inicialización para comparar los alphas.
          torch.manual_seed(SEED)
          np.random.seed(SEED)
          random.seed(SEED)

          agent = CQL(
              state_dim=state_dim,
              action_dim=action_dim,
              max_action=max_action,
              alpha_cql=alpha
          )

          logs = []

          for update in range(1, CQL_UPDATES + 1):
              critic_loss, td_loss, cql_penalty = (
                  agent.train(offline_buffer)
              )

              if update % 10_000 == 0:
                  print(
                      f"Update {update:>6} | "
                      f"Critic {critic_loss:>9.3f} | "
                      f"TD {td_loss:>9.3f} | "
                      f"CQL {cql_penalty:>8.3f}"
                  )

                  logs.append({
                      "update": update,
                      "critic_loss": critic_loss,
                      "td_loss": td_loss,
                      "cql_penalty": cql_penalty
                  })

          cql_agents[alpha] = agent
          cql_training_logs[alpha] = logs

```

===== CQL alpha=0.5 =====

Update	10000	Critic	2.417	TD	0.912	CQL	3.010
Update	20000	Critic	1.871	TD	0.569	CQL	2.603
Update	30000	Critic	2.620	TD	1.329	CQL	2.582

Update	40000		Critic	1.959		TD	0.820		CQL	2.279
Update	50000		Critic	3.681		TD	2.515		CQL	2.333
Update	60000		Critic	2.257		TD	1.069		CQL	2.376
Update	70000		Critic	2.101		TD	1.050		CQL	2.103
Update	80000		Critic	2.224		TD	1.298		CQL	1.852
Update	90000		Critic	2.090		TD	1.046		CQL	2.089
Update	100000		Critic	1.997		TD	1.136		CQL	1.721

===== CQL alpha=1.0 =====

Update	10000		Critic	3.519		TD	1.145		CQL	2.373
Update	20000		Critic	3.218		TD	1.155		CQL	2.063
Update	30000		Critic	3.080		TD	0.905		CQL	2.175
Update	40000		Critic	3.379		TD	1.432		CQL	1.947
Update	50000		Critic	3.446		TD	1.670		CQL	1.776
Update	60000		Critic	3.200		TD	1.178		CQL	2.021
Update	70000		Critic	3.613		TD	1.868		CQL	1.745
Update	80000		Critic	2.768		TD	1.351		CQL	1.417
Update	90000		Critic	3.260		TD	1.658		CQL	1.603
Update	100000		Critic	2.864		TD	1.600		CQL	1.264

===== CQL alpha=5.0 =====

Update	10000		Critic	13.145		TD	5.873		CQL	1.454
Update	20000		Critic	12.155		TD	7.176		CQL	0.996
Update	30000		Critic	9.020		TD	3.880		CQL	1.028
Update	40000		Critic	9.497		TD	5.754		CQL	0.749
Update	50000		Critic	12.523		TD	9.399		CQL	0.625
Update	60000		Critic	13.888		TD	8.141		CQL	1.149
Update	70000		Critic	5.992		TD	3.652		CQL	0.468
Update	80000		Critic	2.156		TD	7.008		CQL	-0.970
Update	90000		Critic	7.275		TD	6.856		CQL	0.084
Update	100000		Critic	-0.860		TD	4.994		CQL	-1.171

```
[40]: cql_results = {}

print(
    f"TD3 subóptimo: "
    f"{suboptimal_mean:.2f} ± {suboptimal_std:.2f}"
)

for alpha, agent in cql_agents.items():
    mean_reward, std_reward = evaluate_policy(
        agent,
        seed=SEED + 100
    )

    cql_results[alpha] = {
        "mean": mean_reward,
```

```

        "std": std_reward
    }

    print(
        f"CQL alpha={alpha:<3}: "
        f"{mean_reward:.2f} ± {std_reward:.2f}"
    )

```

TD3 subóptimo: -163.31 ± 115.00

CQL alpha=0.5: -163.32 ± 114.83

CQL alpha=1.0: -164.44 ± 116.27

CQL alpha=5.0: -164.31 ± 116.17

2.3 Task 2.3

Con base en los resultados de ambos experimentos, respondan:

- Grafiquen las curvas de aprendizaje de TD3 y SAC en la misma figura con media móvil de 10 evaluaciones. ¿Cuál converge más rápido? ¿Cuál alcanza mayor recompensa final? ¿El resultado coincide con la predicción teórica de la Task 1.1b? Si hay discrepancia, argumenten la causa.
- Grafiquen el valor Q promedio durante el entrenamiento para TD3 y SAC. ¿Observan diferencias en la magnitud de los valores estimados? Conéctenlo con el análisis de conservatismo de la Task 1.2b.
- Para el experimento de CQL: ¿alguno de los tres valores de α_{CQL} produce una política que supera a la política subóptima que generó el dataset? ¿Cuál valor de α_{CQL} funciona mejor y por qué? Si ningún valor supera al dataset, argumenten a qué atribuyen ese resultado considerando las características del dataset que generaron.
- Conexión entre paradigmas: con base en los resultados de ambos experimentos, argumenten cuándo recomendarían usar TD3 o SAC versus CQL para el sistema de turbinas eólicas real. Consideren explícitamente: disponibilidad de un simulador confiable, costo de interacción con el sistema real, calidad del dataset histórico disponible, y urgencia de despliegue. Esta pregunta no puede responderse correctamente sin los resultados específicos de sus experimentos.
- Investigación bibliográfica: busquen y lean un paper publicado entre 2023 y 2025 que aplique Offline RL (CQL, BCQ, IQL, o variantes) a un problema de control de sistemas energéticos, robótica industrial, o infraestructura física. El paper debe ser de NeurIPS, ICML, ICLR, IEEE Transactions on Power Systems, o similar. Escriban un resumen técnico de media página con: el problema que resuelve, qué algoritmo de Offline RL usa y por qué, los resultados principales, y una reflexión sobre si las limitaciones del dataset que identificaron en la Task 1.1c también aparecen en ese trabajo y cómo las manejan.

2.3.1 Experimento 1: TD3 vs SAC (Task 2.1)

Los incisos a y b requieren el entrenamiento de la Task 2.1: 100,000 pasos de ambiente por algoritmo, registrando cada 1,000 pasos la recompensa de evaluación greedy (10 episodios), el valor Q promedio sobre un conjunto fijo de 1,000 transiciones y la entropía de la política (solo SAC).

El experimento se ejecuta en esta sección porque reutiliza Actor, Critic, TD3, `soft_update` y `evaluate_policy`, definidos en la Task 2.2. Los registros se guardan en `results/`. Si el archivo ya existe se carga y no se vuelve a entrenar. El entrenamiento de CQL de la Task 2.2 no se vuelve a ejecutar en ningún caso.

Nota sobre la corrida reportada: por tiempo, el entrenamiento se interrumpió antes de completar los 100,000 pasos. TD3 llegó a 90,000 pasos y SAC a 50,000. De esa corrida se conservaron los puntos impresos cada 10,000 pasos (recompensa de evaluación y Q promedio), que son los que se guardan en `results/` y se analizan en los incisos a, b y d. La entropía de SAC y la referencia Monte Carlo solo se guardan al final del entrenamiento, por lo que no están disponibles.

```
[41]: import json
import time
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

RESULTS_DIR = Path("results")
RESULTS_DIR.mkdir(exist_ok=True)

TD3_LOG_PATH = RESULTS_DIR / "td3_online_log.json"
SAC_LOG_PATH = RESULTS_DIR / "sac_online_log.json"
CQL_RESULTS_PATH = RESULTS_DIR / "cql_results.json"

# SAC
SAC_LR = 3e-4
LOG_STD_MIN = -20
LOG_STD_MAX = 2
TARGET_ENTROPY = -float(action_dim)

# Referencia Monte Carlo del valor real
MC_STATES = 100
MC_HORIZON = 500

# Análisis
MOVING_AVG_WINDOW = 10
CONVERGENCE_THRESHOLD = -200
```

```
[42]: class ArrayReplayBuffer:
    # Misma interfaz que ReplayBuffer, pero sobre arreglos de numpy.
    # Indexar un deque es  $O(n)$ , con arreglos el minibatch sale directo.
    def __init__(self, state_dim, action_dim, capacity=BUFFER_SIZE):
        self.capacity = capacity
        self.ptr = 0
        self.size = 0

        self.states = np.zeros((capacity, state_dim), dtype=np.float32)
```

```

self.actions = np.zeros((capacity, action_dim), dtype=np.float32)
self.rewards = np.zeros((capacity, 1), dtype=np.float32)
self.next_states = np.zeros((capacity, state_dim), dtype=np.float32)
self.terminateds = np.zeros((capacity, 1), dtype=np.float32)

def add(self, state, action, reward, next_state, terminated):
    self.states[self.ptr] = state
    self.actions[self.ptr] = action
    self.rewards[self.ptr] = reward
    self.next_states[self.ptr] = next_state
    self.terminateds[self.ptr] = terminated

    self.ptr = (self.ptr + 1) % self.capacity
    self.size = min(self.size + 1, self.capacity)

def sample(self, batch_size=BATCH_SIZE):
    idx = np.random.randint(0, self.size, size=batch_size)

    return tuple(
        torch.as_tensor(array[idx], device=device)
        for array in (
            self.states,
            self.actions,
            self.rewards,
            self.next_states,
            self.terminateds,
        )
    )

def __len__(self):
    return self.size

```

```

[43]: class GaussianActor(nn.Module):
    def __init__(self, state_dim, action_dim, max_action):
        super().__init__()

        self.fc1 = nn.Linear(state_dim, 256)
        self.fc2 = nn.Linear(256, 256)
        self.mean = nn.Linear(256, action_dim)
        self.log_std = nn.Linear(256, action_dim)

        self.max_action = max_action

    def forward(self, state):
        x = F.relu(self.fc1(state))
        x = F.relu(self.fc2(x))

```

```

mean = self.mean(x)
log_std = self.log_std(x).clamp(LOG_STD_MIN, LOG_STD_MAX)

return mean, log_std

def sample(self, state):
    mean, log_std = self(state)
    normal = torch.distributions.Normal(mean, log_std.exp())

    # Reparametrización:  $u = mean + std * eps$ 
    u = normal.rsample()
    y = torch.tanh(u)
    action = self.max_action * y

    # Corrección del log-prob por el cambio de variable tanh y la escala
    log_prob = normal.log_prob(u) - torch.log(
        self.max_action * (1.0 - y.pow(2)) + 1e-6
    )
    log_prob = log_prob.sum(dim=1, keepdim=True)

    greedy_action = self.max_action * torch.tanh(mean)

    return action, log_prob, greedy_action

```

```

[44]: class SAC:
    def __init__(self, state_dim, action_dim, max_action):
        # Actor Gaussiano
        self.actor = GaussianActor(
            state_dim, action_dim, max_action
        ).to(device)

        self.actor_optimizer = optim.Adam(
            self.actor.parameters(),
            lr=SAC_LR
        )

        # Twin Critics y Target Critics
        self.critic = Critic(
            state_dim, action_dim
        ).to(device)

        self.critic_target = Critic(
            state_dim, action_dim
        ).to(device)

        self.critic_target.load_state_dict(
            self.critic.state_dict()

```

```

    )

    self.critic_optimizer = optim.Adam(
        self.critic.parameters(),
        lr=SAC_LR
    )

    # alpha automático: se optimiza log(alpha) para mantenerlo positivo
    self.target_entropy = TARGET_ENTROPY
    self.log_alpha = torch.zeros(
        1, requires_grad=True, device=device
    )

    self.alpha_optimizer = optim.Adam(
        [self.log_alpha],
        lr=SAC_LR
    )

    @property
    def alpha(self):
        return self.log_alpha.exp().item()

    def _to_tensor(self, state):
        return torch.as_tensor(
            state,
            dtype=torch.float32,
            device=device
        ).unsqueeze(0)

    def select_action(self, state):
        # Acción greedy: media de la gaussiana pasada por tanh
        with torch.no_grad():
            _, _, action = self.actor.sample(self._to_tensor(state))

        return action.cpu().numpy()[0]

    def explore_action(self, state):
        with torch.no_grad():
            action, _, _ = self.actor.sample(self._to_tensor(state))

        return action.cpu().numpy()[0]

    def train(self, replay_buffer):
        states, actions, rewards, next_states, terminateds = (
            replay_buffer.sample(BATCH_SIZE)
        )

```

```

alpha = self.log_alpha.exp().detach()

# -----
# Target del Critic (valor suave)
# -----
with torch.no_grad():
    next_actions, next_log_prob, _ = self.actor.sample(
        next_states
    )

    target_q1, target_q2 = self.critic_target(
        next_states,
        next_actions
    )

    target_q = (
        torch.min(target_q1, target_q2)
        - alpha * next_log_prob
    )

    target = rewards + (
        GAMMA
        * (1.0 - terminateds)
        * target_q
    )

# -----
# Actualización Twin Critics
# -----
current_q1, current_q2 = self.critic(
    states,
    actions
)

critic_loss = (
    F.mse_loss(current_q1, target)
    + F.mse_loss(current_q2, target)
)

self.critic_optimizer.zero_grad()
critic_loss.backward()
self.critic_optimizer.step()

# -----
# Actor
# -----
new_actions, log_prob, _ = self.actor.sample(states)

```

```

q1, q2 = self.critic(states, new_actions)

actor_loss = (
    alpha * log_prob
    - torch.min(q1, q2)
).mean()

self.actor_optimizer.zero_grad()
actor_loss.backward()
self.actor_optimizer.step()

# -----
# Ajuste automático de alpha
# -----
alpha_loss = -(
    self.log_alpha
    * (log_prob.detach() + self.target_entropy)
).mean()

self.alpha_optimizer.zero_grad()
alpha_loss.backward()
self.alpha_optimizer.step()

# Target Networks
soft_update(
    self.critic,
    self.critic_target
)

```

El conjunto fijo de 1,000 transiciones se recolecta una sola vez con acciones aleatorias (20 episodios de 50 pasos con semillas distintas) y es el mismo para TD3 y SAC. Sobre esos estados se registra $\min(Q_1, Q_2)$ evaluado en la acción greedy de la política actual, que es la estimación de valor que cada algoritmo usa en su target.

Si el entrenamiento llega al final, se calcula además el retorno descontado real de la política greedy desde los primeros 100 estados del conjunto fijo (horizonte de 500 pasos, $\gamma^{500} \approx 0.007$) como referencia para medir el sesgo del crítico. El Q de SAC es un valor suave e incluye el término de entropía, mientras que el retorno real solo contiene recompensa.

```

[45]: fixed_env = gym.make("Pendulum-v1")
fixed_env.action_space.seed(SEED + 300)

fixed_states = []

for episode in range(FIXED_Q_SAMPLES // 50):
    state, _ = fixed_env.reset(seed=SEED + 300 + episode)

```

```

    for _ in range(50):
        fixed_states.append(state)
        action = fixed_env.action_space.sample()
        state, _, _, _ = fixed_env.step(action)

fixed_env.close()

fixed_states = np.asarray(fixed_states, dtype=np.float32)
fixed_states_t = torch.as_tensor(fixed_states, device=device)

print("Conjunto fijo:", fixed_states.shape)

```

Conjunto fijo: (1000, 3)

```

[46]: def greedy_actions(agent, states):
    if isinstance(agent, SAC):
        return agent.actor.sample(states)[2]

    return agent.actor(states)

def estimate_q(agent, states=fixed_states_t):
    with torch.no_grad():
        q1, q2 = agent.critic(states, greedy_actions(agent, states))

    return torch.min(q1, q2).mean().item()

def policy_entropy(agent, states=fixed_states_t):
    # Estimación Monte Carlo de  $H = -E[\log \pi(a/s)]$  de la política con tanh
    with torch.no_grad():
        _, log_prob, _ = agent.actor.sample(states)

    return -log_prob.mean().item()

def mc_value(agent, states=fixed_states[:MC_STATES], horizon=MC_HORIZON):
    # Retorno descontado real de la política greedy desde cada estado
    mc_env = gym.make("Pendulum-v1").unwrapped
    mc_env.reset(seed=SEED + 400)

    returns = []

    for obs in states:
        # obs = [cos(theta), sin(theta), theta_dot] -> estado [theta, theta_dot]
        mc_env.state = np.array(
            [np.arctan2(obs[1], obs[0]), obs[2]],

```

```

        dtype=np.float64
    )

    state = obs
    discounted_return = 0.0
    discount = 1.0

    for _ in range(horizon):
        state, reward, _, _, _ = mc_env.step(
            agent.select_action(state)
        )

        discounted_return += discount * reward
        discount *= GAMMA

    returns.append(discounted_return)

mc_env.close()

return float(np.mean(returns))

def td3_explore(agent, state):
    action = agent.select_action(state)

    noise = np.random.normal(
        0,
        EXPLORATION_NOISE,
        size=action_dim
    )

    return np.clip(
        action + noise,
        action_low,
        action_high
    ).astype(np.float32)

def sac_explore(agent, state):
    return agent.explore_action(state)

```

```

[47]: def run_online_training(name, make_agent, explore_fn, log_path):
    if log_path.exists():
        print(f"{name}: se carga el registro guardado en {log_path}")
        return None, json.loads(log_path.read_text(encoding="utf-8"))

    random.seed(SEED)

```

```

np.random.seed(SEED)
torch.manual_seed(SEED)

agent = make_agent()
buffer = ArrayReplayBuffer(state_dim, action_dim)

train_env = gym.make("Pendulum-v1")
train_env.action_space.seed(SEED)
state, _ = train_env.reset(seed=SEED)

log = {
    "step": [],
    "eval_mean": [],
    "eval_std": [],
    "q_mean": [],
    "entropy": [],
    "alpha": [],
}

start = time.time()

for step in range(1, TOTAL_STEPS + 1):

    if step <= LEARNING_STARTS:
        action = train_env.action_space.sample()
    else:
        action = explore_fn(agent, state)

    next_state, reward, terminated, truncated, _ = (
        train_env.step(action)
    )

    # Solo terminated corta el bootstrap.
    buffer.add(
        state,
        action,
        reward,
        next_state,
        terminated
    )

    state = next_state

    if step > LEARNING_STARTS:
        agent.train(buffer)

    if terminated or truncated:

```

```

state, _ = train_env.reset()

if step % EVAL_INTERVAL == 0:
    eval_mean, eval_std = evaluate_policy(agent)

    log["step"].append(step)
    log["eval_mean"].append(float(eval_mean))
    log["eval_std"].append(float(eval_std))
    log["q_mean"].append(estimate_q(agent))

    if isinstance(agent, SAC):
        log["entropy"].append(policy_entropy(agent))
        log["alpha"].append(agent.alpha)

    if step % 10_000 == 0:
        print(
            f"{name} | "
            f"Step {step:>6} | "
            f"Eval {eval_mean:>8.2f} | "
            f"Q {log['q_mean'][-1]:>8.2f} | "
            f"{(time.time() - start) / 60:.1f} min"
        )

train_env.close()

log["q_final"] = estimate_q(agent, fixed_states_t[:MC_STATES])
log["mc_final"] = mc_value(agent)
log["minutes"] = (time.time() - start) / 60

log_path.write_text(json.dumps(log, indent=2), encoding="utf-8")
print(f"{name}: registro guardado en {log_path}")

return agent, log

```

```

[48]: td3_online, td3_log = run_online_training(
    "TD3",
    lambda: TD3(state_dim, action_dim, max_action),
    td3_explore,
    TD3_LOG_PATH
)

```

TD3: se carga el registro guardado en results/td3_online_log.json

```

[49]: sac_online, sac_log = run_online_training(
    "SAC",
    lambda: SAC(state_dim, action_dim, max_action),
    sac_explore,
    SAC_LOG_PATH
)

```

```
)
```

SAC: se carga el registro guardado en results/sac_online_log.json

2.3.2 Inciso a

Curvas de aprendizaje con los puntos registrados cada 10,000 pasos. Con esa resolución no hay 10 evaluaciones dentro de una ventana útil, así que la media móvil de 10 evaluaciones que pide el enunciado no se puede calcular y se grafican los puntos directamente. La tabla compara ambos algoritmos en el rango común de 10,000 a 50,000 pasos y reporta el último registro de cada uno.

```
[50]: online_logs = {"TD3": td3_log, "SAC": sac_log}
      colors = {"TD3": "tab:blue", "SAC": "tab:orange"}

      suboptimal_reference = json.loads(
          CQL_RESULTS_PATH.read_text(encoding="utf-8")
      )["suboptimal"]["mean"]

      common_steps = sorted(
          set(td3_log["step"]) & set(sac_log["step"])
      )

      def values_in(log, key, steps):
          lookup = dict(zip(log["step"], log[key]))
          return np.array([lookup[step] for step in steps])

      fig, ax = plt.subplots(figsize=(10, 5))

      for name, log in online_logs.items():
          ax.plot(
              log["step"], log["eval_mean"],
              color=colors[name], marker="o", linewidth=2, label=name
          )

      ax.axhline(
          suboptimal_reference,
          color="gray", linestyle="--", linewidth=1,
          label="TD3 subóptimo (20k pasos)"
      )

      ax.set_xlabel("Pasos de ambiente")
      ax.set_ylabel("Recompensa de evaluación (10 episodios)")
      ax.set_title("Curvas de aprendizaje de TD3 y SAC en Pendulum-v1")
      ax.grid(alpha=0.3)
      ax.legend()
```

```

plt.tight_layout()
plt.show()

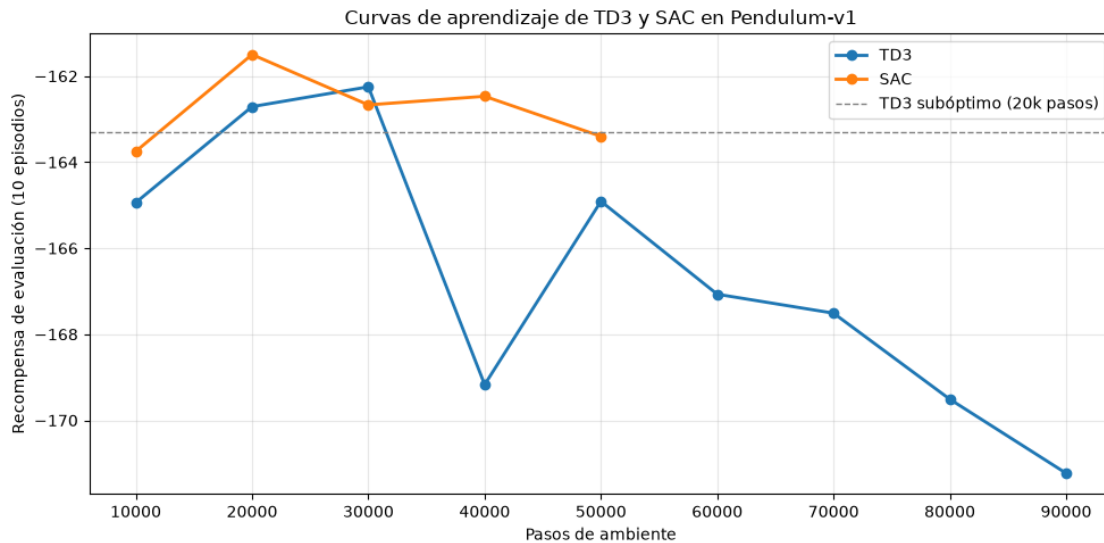
learning_summary = []

for name, log in online_logs.items():
    common = values_in(log, "eval_mean", common_steps)

    learning_summary.append({
        "Algoritmo": name,
        "Pasos registrados": log["step"][-1],
        "Recompensa a 10k pasos": log["eval_mean"][0],
        f"Media {common_steps[0] // 1000}k-{common_steps[-1] // 1000}k": common.
        ↪mean(),
        f"Rango {common_steps[0] // 1000}k-{common_steps[-1] // 1000}k": common.
        ↪max() - common.min(),
        "Último registro": log["eval_mean"][-1],
        "Minutos por 10k pasos": log["minutes"] / (log["step"][-1] / 10_000),
    })

pd.DataFrame(learning_summary).set_index("Algoritmo").round(2)

```



[50]:

	Pasos registrados	Recompensa a 10k pasos	Media 10k-50k \
Algoritmo			
TD3	90000	-164.94	-164.80
SAC	50000	-163.75	-162.76

	Rango 10k-50k	Último registro	Minutos por 10k pasos

Algoritmo			
TD3	6.92	-171.23	0.99
SAC	2.25	-163.40	1.58

Respuesta

Los dos algoritmos convergen antes del primer registro. A los 10,000 pasos TD3 ya obtiene -164.94 y SAC -163.75, prácticamente el mismo nivel que mantienen el resto del entrenamiento y el mismo que la política subóptima de la Task 2.2 (-163.31). Con registros cada 10,000 pasos no se puede separar cuál converge primero. Lo que sí se puede afirmar es que ambos resuelven Pendulum-v1 en menos de 10,000 pasos de ambiente.

SAC alcanza la mayor recompensa. En el rango común de 10,000 a 50,000 pasos su media es -162.76 contra -164.80 de TD3, y su curva es más estable: SAC oscila en un rango de 2.25 puntos y TD3 en uno de 6.92. Además TD3 se degrada después de los 50,000 pasos y termina en -171.23 a los 90,000 pasos, mientras SAC se mantiene en -163.40 en su último registro a 50,000 pasos. El costo de SAC está en el tiempo de cómputo: tarda 1.58 minutos por cada 10,000 pasos contra 0.99 de TD3, porque en cada actualización también entrena el actor y el coeficiente α .

El resultado coincide con la predicción de la Task 1.1b en la dirección, pero no en la magnitud. Predijimos que SAC sería más eficiente en datos y más robusto. La mayor estabilidad de SAC y la degradación de TD3 al final apoyan la parte de robustez: el ruido de exploración fijo de TD3 sigue empujando la política y su crítico, mientras que SAC reduce su exploración de forma automática. La ventaja en eficiencia de datos no se observa. La causa es el entorno: Pendulum-v1 tiene una acción de una dimensión, dinámica suave y un óptimo fácil de encontrar, así que ambos métodos llegan al techo antes de que la diferencia en exploración importe. A eso se suma que es una sola semilla y que la corrida se interrumpió antes de los 100,000 pasos, por lo que la diferencia de 2 puntos entre ambos es pequeña frente a la variabilidad entre episodios (desviación estándar cercana a 115 en la evaluación).

2.3.3 Inciso b

Valor Q promedio, $\min(Q_1, Q_2)$ en la acción greedy, sobre el conjunto fijo de 1,000 estados en cada punto registrado.

```
[51]: fig, ax = plt.subplots(figsize=(10, 5))

for name, log in online_logs.items():
    ax.plot(
        log["step"], log["q_mean"],
        color=colors[name], marker="o", linewidth=2, label=name
    )

ax.set_xlabel("Pasos de ambiente")
ax.set_ylabel(r"$\min(Q_1, Q_2)(s, \pi(s))$ promedio")
ax.set_title("Valor Q promedio sobre el conjunto fijo")
ax.grid(alpha=0.3)
ax.legend()
```

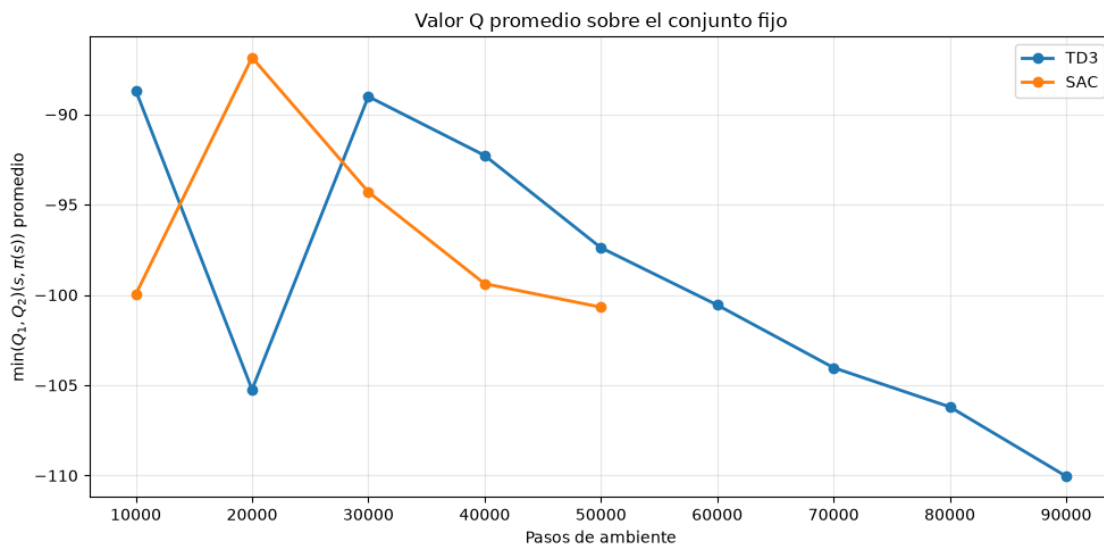
```

plt.tight_layout()
plt.show()

q_summary = pd.DataFrame([
    {
        "Algoritmo": name,
        f"Q medio {common_steps[0] // 1000}k-{common_steps[-1] // 1000}k": (
            values_in(log, "q_mean", common_steps).mean()
        ),
        "Q mínimo": min(log["q_mean"]),
        "Q máximo": max(log["q_mean"]),
        "Último Q": log["q_mean"][-1],
    }
    for name, log in online_logs.items()
]).set_index("Algoritmo")

q_summary.round(2)

```



```

[51]:
      Q medio 10k-50k  Q mínimo  Q máximo  Último Q
Algoritmo
TD3                -94.52   -110.04   -88.68   -110.04
SAC                -96.24   -100.68   -86.86   -100.68

```

Respuesta

Las magnitudes son muy parecidas. En el rango común de 10,000 a 50,000 pasos el Q medio de TD3 es -94.52 y el de SAC -96.24. En casi todos los puntos el Q de SAC queda igual o por debajo del de TD3, por ejemplo -100.68 contra -97.39 a los 50,000 pasos. La excepción es el registro de 20,000 pasos, donde TD3 cae a -105.24. Después de los 50,000 pasos el Q de TD3 baja de forma

sostenida hasta -110.04 a los 90,000 pasos, en paralelo con la caída de su recompensa de evaluación del inciso a.

En la Task 1.2b predijimos que TD3 daría las estimaciones más conservadoras, porque SAC suma al target el término $-\alpha \log \pi_{\theta}(\tilde{a}' | s')$, que en valor esperado es $\alpha \mathcal{H}$ y se esperaba que actuara como bono. Los resultados no muestran ese bono. La explicación más probable está en el ajuste automático de α . La entropía objetivo es $-\dim(A) = -1$, y con acciones continuas la entropía diferencial de una gaussiana angosta es negativa. Cuando el ajuste lleva a la política cerca de esa entropía objetivo, el término $\alpha \mathcal{H}$ vale aproximadamente $-\alpha$ y resta en vez de sumar. Además α se reduce conforme la política se vuelve precisa, así que el término pesa cada vez menos. El resultado es que el crítico suave de SAC queda al nivel del de TD3 o ligeramente por debajo. La conservatividad que dominó en ambos es la que comparten: el mínimo de dos críticos.

La conclusión de la Task 1.2b sobre el dominio se mantiene. En la turbina importa más la conservatividad del mínimo de dos críticos que la diferencia entre algoritmos, y en este experimento ninguno de los dos mostró valores Q que crecieran sin control. La caída sostenida del Q de TD3 al final, acompañada de una peor recompensa, muestra el lado negativo que señalamos: un crítico que se vuelve más pesimista también acompaña a una política que empeora.

2.3.4 Inciso c

Los resultados de CQL se toman del kernel si sigue activo. Si el kernel se reinició, se cargan desde `results/cql_results.json`, que contiene los valores impresos en la Task 2.2. En ningún caso se vuelve a entrenar CQL.

```
[52]: if "cql_results" in globals() and "cql_training_logs" in globals():
    cql_summary = {
        "source": "kernel",
        "suboptimal": {
            "mean": float(suboptimal_mean),
            "std": float(suboptimal_std),
        },
        "dataset_mean": float(np.mean(dataset_episode_rewards)),
        "cql": {
            str(alpha): {
                "mean": float(result["mean"]),
                "std": float(result["std"]),
                "logs": cql_training_logs[alpha],
            }
            for alpha, result in cql_results.items()
        },
    }

    CQL_RESULTS_PATH.write_text(
        json.dumps(cql_summary, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )
else:
    cql_summary = json.loads(
```

```

        CQL_RESULTS_PATH.read_text(encoding="utf-8")
    )

    print("Fuente de resultados CQL:", cql_summary["source"])

    suboptimal_ref = cql_summary["suboptimal"]

    cql_table = pd.DataFrame(
        [{
            "Política": "TD3 subóptimo (genera el dataset)",
            "Media": suboptimal_ref["mean"],
            "Std": suboptimal_ref["std"],
        }]
        + [
            {
                "Política": f"CQL alpha={alpha}",
                "Media": result["mean"],
                "Std": result["std"],
            }
            for alpha, result in cql_summary["cql"].items()
        ]
    ).set_index("Política")

    cql_table["Error estándar"] = cql_table["Std"] / np.sqrt(EVAL_EPISODES)
    cql_table["Diferencia vs subóptimo"] = (
        cql_table["Media"] - suboptimal_ref["mean"]
    )

    print(
        "Recompensa media del dataset (con ruido N(0, 0.3)):",
        round(cql_summary["dataset_mean"], 2)
    )

    cql_table.round(2)

```

Fuente de resultados CQL: outputs del notebook (Task 2.2)

Recompensa media del dataset (con ruido N(0, 0.3)): -159.81

[52]:	Media	Std	Error estándar	\
Política				
TD3 subóptimo (genera el dataset)	-163.31	115.00		36.37
CQL alpha=0.5	-163.32	114.83		36.31
CQL alpha=1.0	-164.44	116.27		36.77
CQL alpha=5.0	-164.31	116.17		36.74
				Diferencia vs subóptimo
Política				

TD3 subóptimo (genera el dataset)	0.00
CQL alpha=0.5	-0.01
CQL alpha=1.0	-1.13
CQL alpha=5.0	-1.00

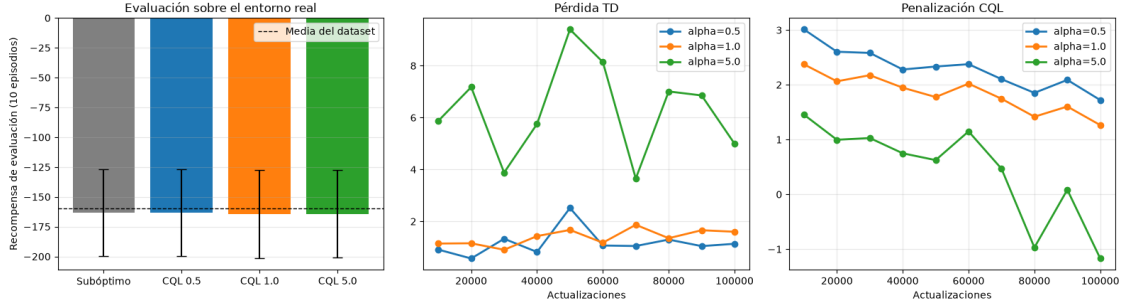
```
[53]: fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

ax = axes[0]
ax.bar(
    range(len(cql_table)),
    cql_table["Media"],
    yerr=cql_table["Error estándar"],
    color=["gray", "tab:blue", "tab:orange", "tab:green"],
    capsize=5
)
ax.axhline(
    cql_summary["dataset_mean"],
    color="black", linestyle="--", linewidth=1,
    label="Media del dataset"
)
ax.set_xticks(range(len(cql_table)))
ax.set_xticklabels(
    ["Subóptimo"] + [f"CQL {a}" for a in cql_summary["cql"]]
)
ax.set_ylabel("Recompensa de evaluación (10 episodios)")
ax.set_title("Evaluación sobre el entorno real")
ax.grid(alpha=0.3, axis="y")
ax.legend()

for metric, ax, title in [
    ("td_loss", axes[1], "Pérdida TD"),
    ("cql_penalty", axes[2], "Penalización CQL"),
]:
    for alpha, result in cql_summary["cql"].items():
        logs = pd.DataFrame(result["logs"])
        ax.plot(
            logs["update"], logs[metric],
            marker="o", linewidth=2, label=f"alpha={alpha}"
        )

    ax.set_xlabel("Actualizaciones")
    ax.set_title(title)
    ax.grid(alpha=0.3)
    ax.legend()

plt.tight_layout()
plt.show()
```



Respuesta

Ninguno de los tres valores supera a la política subóptima. CQL con $\alpha_{CQL} = 0.5$ obtiene -163.32, con $\alpha_{CQL} = 1.0$ obtiene -164.44 y con $\alpha_{CQL} = 5.0$ obtiene -164.31, contra -163.31 de la política que generó el dataset. La mayor diferencia es de 1.13 puntos, mientras que el error estándar de cada evaluación es cercano a 36. En la práctica las cuatro políticas son la misma. Con $\alpha_{CQL} = 0.5$ la coincidencia llega a la centésima.

El valor que funciona mejor es $\alpha_{CQL} = 0.5$, aunque la ventaja no es significativa. Es el menos conservador y el que mejor ajusta la pérdida TD: al final del entrenamiento su pérdida TD es 1.14, contra 1.60 con $\alpha_{CQL} = 1.0$ y 4.99 con $\alpha_{CQL} = 5.0$. Con $\alpha_{CQL} = 5.0$ la penalización domina la pérdida del crítico. La pérdida TD oscila entre 3.65 y 9.40 durante el entrenamiento y la penalización termina negativa (-1.17), lo que indica que el crítico se deformó para separar las acciones del dataset de las aleatorias a costa de ajustar mal la ecuación de Bellman. Aun así la política no cambia, porque el máximo del crítico sigue en el centro de las acciones del dataset.

Atribuimos el resultado a tres características del dataset que generamos:

- La política de comportamiento ya era casi óptima. El TD3 de 20,000 pasos obtiene -163.31, y en el inciso a los entrenamientos de TD3 y SAC se quedan entre -161.50 y -171.23. No había margen de mejora que CQL pudiera aprovechar.
- El dataset viene de una sola política determinista más ruido gaussiano $\mathcal{N}(0, 0.3)$ con media cero. Las acciones de cada estado forman una banda centrada en la acción del TD3 subóptimo. Es lo contrario de la mezcla heterogénea de operadores de la Task 1.1c: no hay trayectorias de un operador mejor que el resto que CQL pueda combinar.
- El ruido no aporta acciones sistemáticamente mejores. Las desviaciones son simétricas y aleatorias, así que en promedio empeoran la acción en lugar de mejorarla. CQL sube el valor de las acciones del dataset y baja el de las que quedan fuera, y la acción con mayor valor termina siendo el centro de la banda, es decir, la acción de la política subóptima. CQL se comportó como una clonación de la política que generó los datos.

Este resultado confirma lo que argumentamos en la Task 1.1c: CQL no puede ir más allá de lo que contienen los datos. Para que supere al dataset hacen falta datos con calidad heterogénea y una política de comportamiento claramente mejorable.

2.3.5 Inciso d

Resumen de ambos experimentos con la cantidad de interacción con el entorno que necesitó cada política.

```
[54]: best_alpha, best_cql = max(
    cql_summary["cql"].items(),
    key=lambda item: item[1]["mean"]
)

final_summary = pd.DataFrame([
    {
        "Paradigma": "Online",
        "Política": f"TD3 ({td3_log['step'][-1] // 1000}k pasos)",
        "Recompensa (último registro)": td3_log["eval_mean"][-1],
        "Pasos de entorno en entrenamiento": td3_log["step"][-1],
    },
    {
        "Paradigma": "Online",
        "Política": f"SAC ({sac_log['step'][-1] // 1000}k pasos)",
        "Recompensa (último registro)": sac_log["eval_mean"][-1],
        "Pasos de entorno en entrenamiento": sac_log["step"][-1],
    },
    {
        "Paradigma": "Online (generador del dataset)",
        "Política": "TD3 subóptimo (20k pasos)",
        "Recompensa (último registro)": suboptimal_ref["mean"],
        "Pasos de entorno en entrenamiento": SUBOPTIMAL_STEPS,
    },
    {
        "Paradigma": "Offline",
        "Política": f"CQL alpha={best_alpha} (mejor)",
        "Recompensa (último registro)": best_cql["mean"],
        "Pasos de entorno en entrenamiento": 0,
    },
]).set_index("Política")

final_summary.round(2)
```

```
[54]:
```

	Paradigma \
Política	
TD3 (90k pasos)	Online
SAC (50k pasos)	Online
TD3 subóptimo (20k pasos)	Online (generador del dataset)
CQL alpha=0.5 (mejor)	Offline

	Recompensa (último registro) \
Política	
TD3 (90k pasos)	-171.23
SAC (50k pasos)	-163.40
TD3 subóptimo (20k pasos)	-163.31
CQL alpha=0.5 (mejor)	-163.32

Pasos de entorno en entrenamiento

Política	
TD3 (90k pasos)	90000
SAC (50k pasos)	50000
TD3 subóptimo (20k pasos)	20000
CQL alpha=0.5 (mejor)	0

Respuesta

Los dos experimentos llegaron al mismo nivel de desempeño por caminos distintos. TD3 y SAC alcanzaron cerca de -163 en menos de 10,000 pasos de interacción. CQL obtuvo -163.32 sin interactuar con el entorno, pero nunca superó a la política que generó sus datos, y esos datos ya habían costado 20,000 pasos de entrenamiento más 50,000 de recolección. La elección entre paradigmas depende de cuatro factores.

- Disponibilidad de un simulador confiable. Si existe, recomendamos SAC entrenado en simulación. En nuestros resultados SAC fue el más estable y el que alcanzó la mejor recompensa media. En Pendulum ambos llegaron al techo en menos de 10,000 pasos. La turbina necesitará muchos más, pero la eficiencia de los métodos off-policy con replay buffer hace viable el aprendizaje en línea aun con un simulador aeroelástico lento. Si el simulador no es confiable, la política en línea aprende a explotar sus errores y CQL sobre datos reales es la opción más segura.
- Costo de interacción con el sistema real. Explorar sobre una turbina real no es aceptable: en los primeros episodios de entrenamiento de TD3 en la Task 2.2 la recompensa rondaba -1,400, que en una turbina equivale a cargas y paros de emergencia. Si no hay simulador, CQL es la única opción de las dos, porque entrega una política con cero interacciones.
- Calidad del dataset histórico. Nuestro CQL reprodujo la política de comportamiento pero no la mejoró. Si los registros vienen de operadores expertos, CQL entrega rápido y con poco riesgo un controlador al nivel de esos expertos. Si los registros son pobres o poco diversos, CQL queda limitado a ese nivel y hace falta mejorar la política en simulación. La heterogeneidad de los operadores de la Task 1.1c ayuda en este punto, porque da a CQL comportamientos buenos que combinar, algo que nuestro dataset de una sola política no tenía.
- Urgencia de despliegue. CQL solo necesita cómputo sobre datos que ya existen y no exige construir ni validar un simulador. En nuestro caso entrenar los tres valores de α_{CQL} tomó cerca de 80 minutos. Con urgencia recomendamos desplegar primero una política CQL con α_{CQL} alto, que se mantiene cerca de lo que hacían los operadores. Sin urgencia y con simulador, vale la pena invertir en SAC para buscar un desempeño mejor que el humano.

Nuestra recomendación para el sistema real combina ambos paradigmas. Primero CQL sobre los tres años de registros, para tener sin riesgo una política al nivel de los mejores operadores. Después SAC en un simulador validado contra esos mismos registros, partiendo de la política CQL, para mejorarla. Al desplegar se usa la acción media de SAC, como se argumentó en la Task 1.1b.

2.3.6 Inciso e

Respuesta

Paper seleccionado: *Offline Reinforcement Learning for Microgrid Voltage Regulation* (Yang y Zhu, 2025), presentado en ICLR 2025.

Problema que resuelve El trabajo aborda el control de voltaje en microrredes con alta penetración de energía solar fotovoltaica (PV), un problema crítico en sistemas de distribución modernos donde la generación renovable intermitente y la demanda variable pueden causar inestabilidad de voltaje. La regulación tradicional requiere interacción en tiempo real con el entorno, lo cual es costoso, riesgoso y potencialmente desestabilizador en sistemas eléctricos reales. El objetivo es aprender una política de control óptima sin interacción en línea, usando únicamente datos históricos pre-colectados.

Algoritmo de Offline RL usado y justificación Los autores comparan dos algoritmos de Offline RL:

- BCQ (Batch-Constrained Q-Learning): restringe las acciones del agente a un conjunto cercano a las observadas en el dataset mediante un modelo generativo (VAE) condicionado al estado, mitigando el error de extrapolación en pares estado-acción no vistos.
- CQL (Conservative Q-Learning): introduce un término de penalización conservadora en la función de pérdida de la Q-function que subestima sistemáticamente los valores de acciones fuera de distribución (OOD), evitando sobreestimaciones peligrosas.

Ambos están diseñados específicamente para el régimen offline, donde el agente no puede explorar activamente y debe evitar acciones no representadas en el dataset. CQL es particularmente atractivo en aplicaciones de seguridad crítica, como sistemas de potencia, porque su conservadurismo en la estimación de valores reduce el riesgo de políticas inestables.

Resultados principales Los experimentos se realizaron en el sistema IEEE 33-bus con tres datasets de calidad variable: Expert (política DDPG bien entrenada), Medium (ruido gaussiano de 5% sobre Expert) y Poor (acciones aleatorias).

- CQL superó consistentemente a BCQ en los tres datasets, especialmente en escenarios de baja y media calidad de datos.
- En el dataset Poor, CQL mantuvo ratios de voltaje totalmente controlable más altos y desviaciones de voltaje menores que BCQ.
- En el dataset Expert, ambos algoritmos alcanzaron un rendimiento cercano al óptimo, pero CQL mostró mayor estabilidad.
- Los autores concluyen que CQL es una opción más segura y robusta para regulación de voltaje, gracias a su estimación conservadora de valores Q.

Reflexión sobre limitaciones del dataset (Task 1.1c) Las limitaciones típicas en datasets de Offline RL, como cobertura insuficiente del espacio estado-acción, sesgo hacia políticas subóptimas y falta de diversidad en experiencias, aparecen explícitamente en este trabajo. Los autores las identifican y cuantifican al crear los datasets Expert, Medium y Poor, mostrando estadísticas de recompensa, varianza y retorno promedio por episodio.

Cómo las manejan:

1. Evaluación sistemática: comparan BCQ y CQL en los tres niveles de calidad para entender la sensibilidad de cada algoritmo a la distribución de datos.
2. Conservadurismo algorítmico: CQL penaliza explícitamente acciones OOD, lo que lo hace más robusto frente a datasets con baja cobertura o alto ruido.

3. Generación de datos estructurada: usan un simulador de flujo de potencia (DistFlow) con datos reales de carga y generación PV para garantizar que los datasets reflejen condiciones operativas realistas, aunque limitadas.

Sin embargo, el trabajo no aborda explícitamente limitaciones como el sim-to-real gap (todos los datos son simulados) ni la escalabilidad a redes más grandes, lo cual los autores reconocen como dirección futura. En comparación con la Task 1.1c, este paper maneja mejor la limitación de calidad variable de datos mediante evaluación comparativa y selección de algoritmos robustos, pero comparte la limitación de dependencia de simulación y la falta de validación en sistemas físicos reales.

Referencia Yang, S., y Zhu, Y. (2025). *Offline Reinforcement Learning for Microgrid Voltage Regulation*. Presentado en ICLR 2025. arXiv:2505.09920. <https://arxiv.org/abs/2505.09920>

2.4 Prompts

Siguiendo la nota del laboratorio, el contenido técnico se trabajó con fuentes de internet. La IA generativa se usó solo en dos tareas de soporte para el reporte. Los prompts fueron los siguientes.

2.4.1 Prompt 1: generar el Reporte.pdf

Uso: generar el Reporte.pdf. Aplica a todo el reporte.

Genera el Reporte.pdf usando mi nix-shell -p porque tengo NixOS.

Por qué funcionó: en NixOS no se pueden instalar paquetes con pip de forma global y las librerías del sistema no están en las rutas comunes. Al decir de entrada que el sistema es NixOS y que se use nix-shell -p, la respuesta armó un entorno temporal con nbconvert, pandoc y texlive en un solo comando. Así se generó el PDF sin instalar nada de forma permanente y sin errores de dependencias de LaTeX.

2.4.2 Prompt 2: corregir las fórmulas en markdown

Uso: fórmulas de la Task 1.1 y la Task 1.2.

Prompt:

Corrige las fórmulas del markdown del notebook. No están escritas como en el documento de la tarea. Usa la misma notación que el documento, por ejemplo $\pi(\cdot | S_t)$, $\mathcal{H}$ y α_{CQL} , y escríbelas en LaTeX para que se rendericen bien en el PDF.

Por qué funcionó: el prompt indica el problema concreto, que la notación no coincidía con la del documento, y da ejemplos de los símbolos esperados. Con esos ejemplos la corrección se limitó a la notación sin cambiar el contenido de las respuestas. Pedir LaTeX explícitamente asegura que las fórmulas se vean igual en el notebook y en el PDF generado con nbconvert.